CONTACT

+82-10-2627-1356

dmsgp2627@naver.com

github.com/eeeunhey

EDUCATION

성남 폴리텍 하이테크 과정
2024.01 – 2024.12
Java (Spring/Boot/MyBatis/JPA)
Python (LLM / Object Detection)

KDT 국비 과정 수료
2023.03 – 2023.09
C/Java/Python 기초, 실습

방송통신대(식품영양학)
2016.02 – 2021.02

KEY SKILLS

Java · Python · SQL · Spring · Spring Boot · MyBatis · JPA · React · Vite · TailwindCSS · LLM · Object Detection · Git · Docker · .NET C# WinForms

CERTIFICATIONS

컴퓨터활용능력 1급, 2급

ITQ A등급 (한글, 엑셀, 파워포인트)

GTQ 1급, 2급

자동차운전면허 1종 보통

문제에서 배움을 찾고 성장하는 김은혜 입니다

JUNIOR DEVELOPER (WEB · BACKEND · AI)

PROFESSIONAL PROFILE

저는 새로운 시도와 도전을 통해 문제를 발견하고 해결하며 성장해왔습니다.
작은 단위로 나누어 해결하고 개선을 반복한 경험은 저를 더 나은 방향으로 이끌었고,
앞으로도 이런 태도를 바탕으로 계속 발전해 나가겠습니다.

EXPERIENCE

솔루션팀 · 머신 비전 파트

2024.02 – 2024.11

- 비전 셋팅 및 카메라 파라미터 조정 지원
- C# WinForms UI 일부 수정 및 테스트
- 테스트 결과 정리 및 개선 사항 보고

쿠팡 물류 · 전산 / 관리

2019.03 – 2021.11

- 단기 인력 배치와 스케줄 조율
- 출고 전산 흐름 점검 및 마감 관리
- 마감 시점의 소통·조율 업무

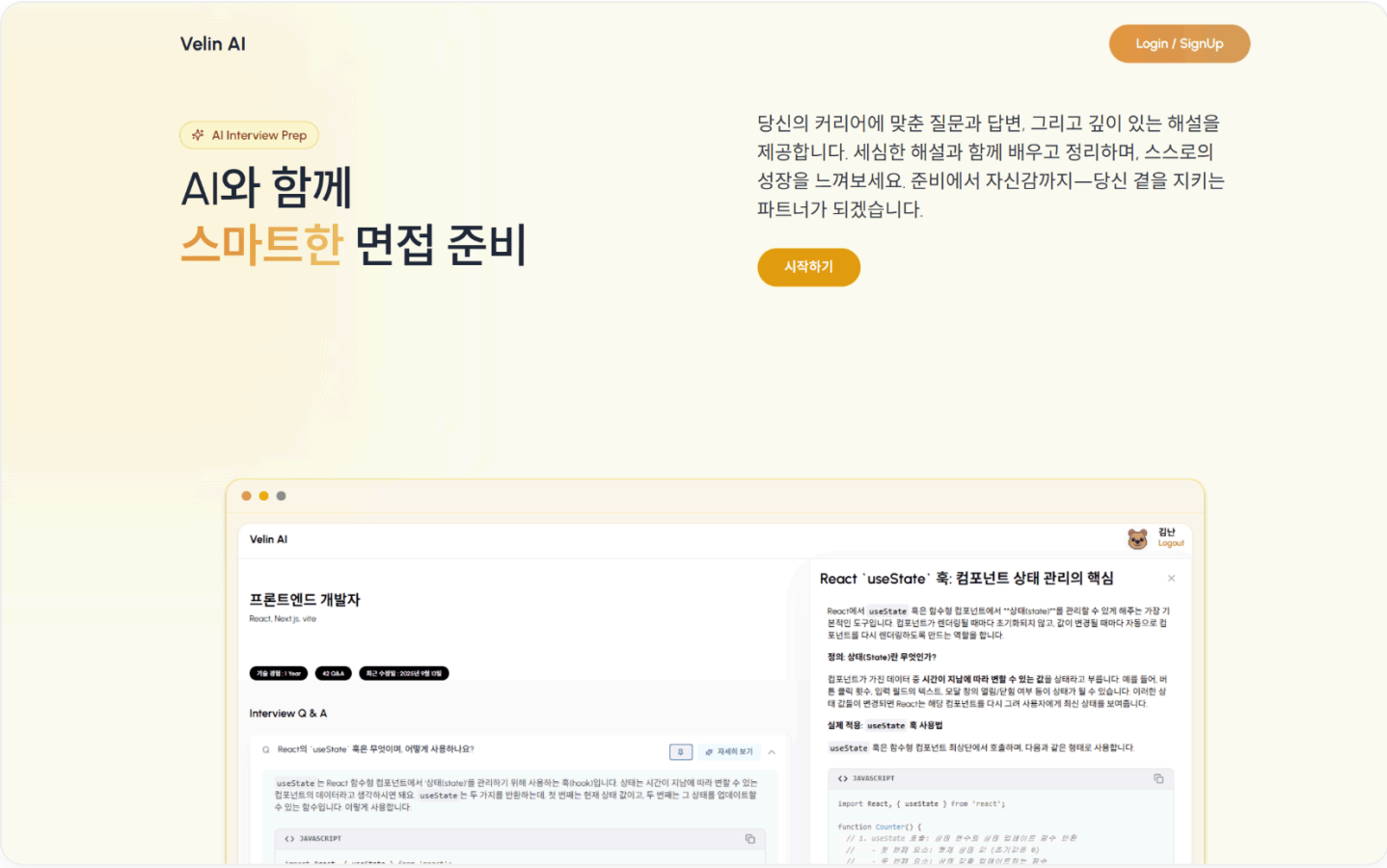
PROJECTS

Velin — AI Interview Prep
2025.08–2025.09 (개인)

ALBAIT — 콘솔 구인·구직 실습
2025.04–2025.05 (개인)

Velin — AI Interview Prep

2025.08–2025.09 (개인) · Vite·React·Tailwind / Node·Express / Recharts / TanStack Query / FFmpeg(WASM)



개요

면접 질문·근거·체크리스트 자동 생성과 멀티모달(영상/텍스트) 분석 기반 세션·질문 단위 리포트를 제공합니다.

개발 기간/인원

2025.08–2025.09 (6주) · 개인

기획 의도

- 이론 학습만으로는 협업·운영 관점에서 부족함을 느꼈습니다
- "함께 일하고 싶은 동료라면 어떤 질문과 근거가 필요할까?"라는 문제의식에서 출발했습니다
- 질문·참고 자료·체크리스트를 한눈에 제공하는 실무형 도구를 기획했습니다

핵심기능

- LLM 기반 질문/질문에 대한 근거 이론 / 코드 예시 제공
- IT 직군별 실무 질문 생성 후 제공
- 직군별 생성 질문 후 질문 추가 생성

기술스택

React · Vite · Tailwind · Node·Express · MongoDB

성과·배운 점

- ✓ **프롬프트 템플릿 설계** 과정을 통해 AI 출력 제어·일관성 유지의 중요성을 배움
- ✓ **React–Express–MongoDB 연동**으로 프론트와 백엔드, DB 연결 구조를 이해
- ✓ **JWT 인증과 파일 업로드**를 적용하며 보안과 데이터 흐름 관리 경험 습득

참고 링크

https://github.com/eeeunhey/AI_InterviewPrep

ALBAIT — 콘솔 구인·구직 사이트 구현

2025.04–2025.05 (개인) · Java · Oracle · JDBC · DAO/VO

개요

개인/기업 회원관리, 공고/지원/상태변경, 관리자 통계를 제공하는 콘솔 기반 프로젝트.

개발 기간/인원

2025.04–2025.05 (개인)

기획 의도

- 단순 CRUD 예제로는 실제 구인·구직 서비스 구조를 충분히 이해하기 어렵다고 느꼈습니다
- 회원·공고·지원 흐름을 직접 설계하여 실제 구직 과정을 콘솔 환경에서 구현했습니다
- DB 설계, DAO/VO, 트랜잭션 처리 등 실무 아키텍처 패턴을 학습하기 위해 프로젝트 진행 하였습니다

핵심기능

- 개인/기업 회원, 공고 등록/지원, 지원 상태 변경
- 관리자 통계(사용자/지원 현황) 조회

기술스택

Java · Oracle · JDBC · DAO/VO

성과·배운 점

✓ **인덱스와 실행계획 분석**을 통해 SQL 성능 최적화 원리 학습

✓ **DAO/Service 분리**로 계층형 아키텍처 설계의 중요성 이해

✓ **예외 처리와 트랜잭션 관리**를 통해 안정적인 데이터 흐름 관리 방법을 습득

✓ **회원–공고–지원 흐름**을 구현하며 실제 구직 서비스 구조를 콘솔 환경에서 모사하는 경험

✓ **데이터베이스 모델링과 DAO/VO 패턴**을 통해 실무형 서비스 설계 감각을 익힘

참고 링크

https://github.com/eeeunhey/AlbaitProject

자기소개서

저는 문제를 마주했을 때 단순히 '왜 안 되는지'를 찾기보다 **'서로 다른 기술과 아이디어를 어떻게 결합하면 더 나은 구조로 발전시킬 수 있을까'**를 고민합니다.
새로운 기술을 배울 때도 단순히 기능을 익히는 데 그치지 않고, 그것이 기존 흐름과 어떤 방식으로 연결될 수 있는지를 직접 실험하며 이해하는 것을 좋아합니다.
이런 탐구의 과정 속에서 문제 상황은 오히려 새로운 조합을 시도할 기회가 되었고, 그 과정에서 개발의 진짜 즐거움을 느꼈습니다.

프로젝트를 진행하면서 단순히 기능이 동작하는 것보다, **"왜 이 문제가 발생했고, 어떻게 구조적으로 개선할 수 있을까"**를 중심으로 접근했습니다.
아래는 그 과정에서 실제로 느낀 문제와 이를 해결하기 위해 적용한 경험들입니다.

🟦 **명확한 타입 정의로 유지보수 속도를 개선**

- React 프로젝트를 진행하며 컴포넌트 간 데이터 전달이 많아지자, **어떤 props가 어디서 넘어오고, 어떤 타입으로 쓰이는지**를 추적하는 데 많은 시간을 쏟게 되었습니다. 데이터 구조가 조금만 바뀌어도 연쇄적으로 오류가 발생해 유지보수가 점점 어려워졌습니다.
- 이런 반복적인 불편함을 줄이기 위해 TypeScript를 도입했습니다. 타입을 명확히 지정하니 IDE에서 **자동완성·오류 탐지·함수 추적**이 가능해졌고, 수정 시 영향 범위를 빠르게 파악할 수 있었습니다.
- 특히 여러 API가 동시에 호출되는 화면에서 **데이터 구조가 바뀌면 즉시 에러가 표시**되어, 런타임 오류보다 훨씬 빠르게 문제를 인식하고 대응할 수 있었습니다.
  단순한 타입 검증을 넘어서, 코드의 흐름이 **눈에 보이는 구조**로 바뀌는 경험이었습니다.

🟩 **Zustand 기반 전역 스토어 리팩터링으로 상태 불일치 개선**

- React 프로젝트가 커지면서 여러 컴포넌트가 각자 상태를 관리하게 되었고, 한쪽의 변경이 다른 화면에 반영되지 않거나 데이터가 어긋나는 문제가 발생했습니다.
- 프로젝트가 커지면서 여러 컴포넌트가 동일한 데이터를 서로 다른 방식으로 관리하면서, **하나의 수정이 다른 화면에 반영되지 않거나 예기치 않은 재렌더링**이 자주 일어났습니다.
- 이를 해결하기 위해 Zustand를 도입해 상태를 **한 곳에서 일관되게 관리**하도록 구조를 변경했습니다. 전역 스토어로 감정·집중도·답변 데이터를 통합하고, 각 컴포넌트는 필요한 값만 구독하도록 분리하니 화면이 안정적으로 동기화되고 **렌더 효율**도 향상되었습니다.
- 현재는 작은 기능부터 점진적으로 Zustand를 확장하며, **단일 소스·명확한 액션·불필요한 렌더 최소화** 원칙을 중심으로 구조를 다듬고 있습니다.

🟥 **DAO/DTO 패턴으로 DB 연동 구조 개선 및 유지보수성 향상**

- 콘솔 기반 **ALBAIT** 프로젝트에서는 게시글 등록과 회원 데이터를 DB와 직접 연동했습니다. 처음에는 SQL 쿼리를 코드 안에서 바로 호출했지만,
  기능이 늘어날수록 의존 관계가 복잡해지고 수정 시 오류가 잦았습니다.
- 이를 해결하기 위해 **DAO/DTO 패턴**을 도입해 **요청 → DB 처리 → 응답** 과정을 단계별로 나눴습니다.
  이전에는 코드 곳곳에서 SQL을 직접 다뤘지만, 지금은 데이터 흐름이 한눈에 보이고 수정할 위치도 명확해졌습니다.
  데이터가 어디서 만들어지고, 어디로 전달되는지 흐름이 뚜렷해지면서 오류를 찾고 수정하는 시간이 크게 줄었습니다.

여러 프로젝트를 하면서, **데이터와 상태는 서로 영향을 주기 때문에 하나의 흐름 안에서 함께 관리해야 한다**는 걸 배웠습니다.
저는 단순히 문제를 해결하는 데서 멈추지 않고, **구조를 분석하고 개선점을 찾아내 실제 성과로 이어지게 하는 과정**에서 보람을 느낍니다.
새로운 기술이나 접근법이 등장할 때마다, 이를 기존 시스템에 **효율적으로 결합해 속도나 유지보수성을 개선하는 방법**을 꾸준히 실험해왔습니다.
작업할 때는 '이 코드가 전체 구조 속에서 어떤 가치를 만들어내는가'를 항상 고려하며, 문제를 부분적으로 보기보다, **전체 흐름 속에서 가장 합리적인 해결 방법**을 찾으려 합니다.
이런 접근을 통해 기능을 안정적으로 개선하고, 복잡한 로직을 단순화해 팀의 효율을 높이기 위해 꾸준히 노력하고 있습니다.

앞으로는 구조적 사고를 기반으로, **성과를 단순히 내는 데 그치지 않고, 그 결과를 분석하고 보완하며 한 단계 더 나은 가치를 만들어가는 개발자**로 성장하고자 합니다.
다양한 기술과 아이디어를 함께 엮어 더 큰 성과를 만들어가는 사람이 되고 싶습니다.